

GRADUATION PROJECT

Design and Simulation of a Secure Authentication System for Smart Home IoT

using RADIUS, LDAP, Kerberos and IEEE 802.1X/EAP in ns-3.41

Source Code and Implementation Guide

Prepared by

Amira Bdeir and Thouraya Sheikha

ns-3 Version	ns-3.41
Implementation Language	C++
Environment	Ubuntu Linux / VMware
Primary Source File	scratch/iot-defense-demo.cc
Document Type	Source Code and Implementation Guide (supporting documentation)

Table of Contents

1. Cover Page
2. Project Overview
3. Final System Architecture
4. Smart-Home Topology
5. Network Configuration and Custom Packet Headers
6. Authentication and Authorization Workflow
7. Gateway Security Pipeline
8. LDAP Cache Evaluation
9. Replay Attack Scenario
10. Privilege Escalation Scenario
11. Build and Execution Commands
12. Code Defense Index
13. Main Code Excerpts

2. Project Overview

This project designs and simulates a multilayer authentication and access-control architecture for a smart-home Internet of Things environment using ns-3.41. The complete implementation is contained in the main simulation file, `scratch/iot-defense-demo.cc`. The simulation models the security workflow that must be completed before an authorized human user or IoT sensor can access the network, obtain the required credentials, and communicate with simulated smart-home devices such as a smart door lock, light, camera, and plug.

The proposed architecture functionally integrates several established security technologies at the ns-3 application layer:

- **IEEE 802.1X, EAP, and EAPOL:** provide network access control between the Supplicant and the Authenticator.
- **RADIUS:** provides centralized Authentication, Authorization, and Accounting (AAA).
- **LDAP and LDAP Cache:** provide identity, account-status, role, permission-mask, and credential-verification information. The optional RADIUS-side cache reduces repeated LDAP queries.
- **Kerberos-inspired ticketing:** issues a ticket only after successful IEEE 802.1X authorization and verification of the corresponding authenticated RADIUS session.

The IEEE 802.1X Authenticator does not independently decide whether access should be granted. It receives EAP messages inside EAPOL from the Supplicant, encapsulates EAP inside a RADIUS request, forwards the request to the RADIUS server, and then enforces the resulting Access-Accept or Access-Reject decision by setting the principal's state to `AUTHORIZED` or `UNAUTHORIZED`.

After successful authorization, the principal requests a Kerberos-inspired ticket. Each subsequent command is sent to the Smart-Home Gateway, which acts as the central security enforcement point. The Gateway validates the ticket, verifies the Command MAC, checks the Sequence Number for replay detection, applies Role-Based Access Control, and forwards the command to the target IoT device only when all required checks succeed.

The implementation includes exactly two security attack scenarios:

- **Replay Attack:** an accepted command packet is captured and resent. Protection is provided through duplicate Sequence Number detection.
- **Privilege Escalation Attack:** the role and permission claims carried in a command are modified in an attempt to obtain unauthorized access. The modification is detected through Command MAC verification.

The simulation also evaluates LDAP Cache behavior, RADIUS Accounting, authentication decisions, ticket issuance, command acceptance and rejection, and the effectiveness of the two implemented protection mechanisms. All architectural descriptions, configuration values, code references, and numerical results presented in this guide are derived from the tested `iot-defense-demo.cc` implementation.

3. Final System Architecture

The project implements the following end-to-end security workflow. Most transitions represent UDP exchanges between ns-3 nodes running dedicated `Application` subclasses. The LDAP Cache is an internal RADIUS-side optimization; when a cache hit occurs, the RADIUS server reuses the cached directory information without sending a new LDAP query.

```

Supplicant
↓ EAP inside EAPOL
IEEE 802.1X Authenticator
↓ EAP encapsulated inside RADIUS
RADIUS Server
↓
LDAP / LDAP Cache
↓
RADIUS Access-Accept or Access-Reject
↓
Authenticator enforces the RADIUS decision (AUTHORIZED / UNAUTHORIZED)
↓ (only if AUTHORIZED)
Kerberos-inspired Ticket (KDC)
↓
Gateway
  1. Ticket Validation
  2. Command MAC Verification
  3. Sequence Number Replay Detection
  4. RBAC Permission Check
  5. Sequence Number Storage
  6. Forwarding to the IoT Device

```

Architectural rules enforced by the implementation

- **EAPOL is used only between the Supplicant and the IEEE 802.1X Authenticator.** The `SmartHomeUserApp` sends an `EapHeader` encapsulated inside an `EapolHeader` to `Dot1xAuthenticatorApp`. EAPOL is not used between the Authenticator and the RADIUS server.
- **EAP is encapsulated inside the RADIUS request between the Authenticator and RADIUS.** In `Dot1xAuthenticatorApp::ReceiveEapol`, the Authenticator removes the EAPOL wrapper, retains the EAP message, and places it inside a packet containing an outer `RadiusAuthHeader` before forwarding it to `RadiusServerApp`.
- **RADIUS makes the authentication and authorization decision.** `RadiusServerApp` verifies the RADIUS authenticator, validates the credential proof through LDAP or the LDAP Cache, checks account status, retrieves the role and permission mask, and produces an Access-Accept or Access-Reject result in `RadiusServerApp::FinishAuth`.
- **The Authenticator enforces the RADIUS decision.** In `Dot1xAuthenticatorApp::ReceiveRadiusReply`, an Access-Accept changes the principal's IEEE 802.1X state to `AUTHORIZED`, while an Access-Reject keeps the state `UNAUTHORIZED`. The Authenticator then returns EAP-Success or EAP-Failure to the Supplicant inside EAPOL.
- **A rejected principal cannot continue to the ticket stage.** `SmartHomeUserApp::ReceiveEapolResult` sends a ticket request only after receiving EAP-Success. Therefore, a principal such as Lina, whose authentication is rejected, does not request or receive a Kerberos-inspired ticket.
- **The KDC issues tickets only to authorized principals associated with a valid authenticated RADIUS session.** Before issuing a ticket, `KdcApp::IssueKerberosTicket` verifies both the IEEE 802.1X authorization state and the session identifier received from the RADIUS server.
- **The Gateway applies the security checks in a fixed order.** `SmartHomeGatewayApp::ProcessGatewayCommand` executes:

Ticket Validation

→ Command MAC Verification

→ Sequence Number Replay Detection

→ RBAC Permission Check

→ Sequence Number Storage

→ Forwarding to the IoT Device

A command is forwarded only when all required checks succeed.

4. Smart-Home Topology

All nodes are connected to a single ns-3 CSMA network configured with a data rate of **10 Mbps** and a delay of **2 ms**. A standard IPv4 stack is installed on all nodes by `CreateNetworkTopology()`.

The simulation contains **17 nodes**, declared in the `NodeIndex` enumeration:

```
N_AUTH, N_RADIUS, N_LDAP, N_KDC, N_GW,
N_DOOR, N_LIGHT, N_PLUG, N_TEMP, N_MOTION,
N_AHMAD, N_SARA, N_OMAR, N_LINA, N_REPLAY,
N_CACHE_PROBE, N_CAMERA
```

4.1 Infrastructure entities

Entity	ns-3 Application Class	Role
IEEE 802.1X Authenticator	Dot1xAuthenticatorApp	Receives EAP inside EAPOL from the Supplicant, encapsulates EAP inside RADIUS, and enforces the RADIUS decision as <code>AUTHORIZED</code> or <code>UNAUTHORIZED</code> .
RADIUS Server	RadiusServerApp	Provides Authentication, Authorization, and Accounting using LDAP or the LDAP Cache.
LDAP Server	LdapServerApp	Stores account status, role, permission mask, and password-hash information.
Kerberos-inspired KDC	KdcApp	Issues tickets only to authorized principals with a valid authenticated RADIUS session.
Smart-Home Gateway	SmartHomeGatewayApp	Policy Enforcement Point for commands and telemetry messages.

4.2 Users (Principals)

Principal	Principal ID	Role	Permission Mask	Enabled	Legitimate Commands
Ahmad	100 (USER_AHMAD)	OWNER	TELEMETRY DOOR LIGHT CAMERA PLUG	Yes	OPEN_DOOR, CAMERA_VIEW
Sara	101 (USER_SARA)	GUEST	LIGHT	Yes	LIGHT_ON; her OPEN_DOOR attempt is denied by RBAC.
Omar	102 (USER_OMAR)	MAINTENANCE	TELEMETRY PLUG	Yes	PLUG_ON

Principal	Principal ID	Role	Permission Mask	Enabled	Legitimate Commands
Lina	103 (USER_LINA)	DISABLED	(none)	No	(none — the account is disabled in LDAP, so RADIUS rejects the authentication request)

The user identities, roles, permissions, and account states are defined in `BuildLdapDirectory()` and `InstallUsers()`.

4.3 IoT entities

Device / Sensor	Device ID	Type	Command
Temperature Sensor	1 (DEV_TEMPERATURE)	Telemetry source (ROLE_DEVICE)	SEND_TEMPERATURE
Motion Sensor	2 (DEV_MOTION)	Telemetry source (ROLE_DEVICE)	SEND_MOTION
Smart Door Lock	3 (DEV_DOOR)	Actuator (SmartDeviceApp)	OPEN_DOOR
Smart Light	4 (DEV_LIGHT)	Actuator (SmartDeviceApp)	LIGHT_ON
Smart Camera	5 (DEV_CAMERA)	Actuator (SmartDeviceApp)	CAMERA_VIEW
Smart Plug	6 (DEV_PLUG)	Actuator (SmartDeviceApp)	PLUG_ON

The Temperature Sensor and Motion Sensor authenticate through the complete IEEE 802.1X, RADIUS, LDAP, and Kerberos-inspired workflow. Their accepted data is reported by the Gateway as: `[TELEMETRY ACCEPTED]`. The Smart Door Lock, Smart Light, Smart Camera, and Smart Plug are actuator nodes. Each runs a `SmartDeviceApp` instance and receives commands only after successful Gateway validation.

5. Network Configuration and Custom Packet Headers

5.1. Network Ports

All communication ports are defined as named constants in `iot-defense-demo.cc` and are used consistently by the corresponding ns-3 applications.

Constant	Port	Purpose
EAPOL_PORT	8880	Supplicant ↔ Authenticator (EAP inside EAPOL)
RADIUS_PORT	1812	Authenticator → RADIUS (RADIUS Authentication, EAP encapsulated)
RADIUS_ACCT_PORT	1813	SmartHomeUserApp → RADIUS (RADIUS Accounting Start / Stop)
LDAP_PORT	3890	RADIUS ↔ LDAP (identity / role / permission / password-hash lookup)
KDC_PORT	8888	Ticket requests and responses between authorized principals and the Kerberos-inspired KDC.

Constant	Port	Purpose
GATEWAY_PORT	9000	Supplicant / ReplayAttackApp → Gateway (Ticket + Command)
DEVICE_PORT	9100	Gateway → IoT Device (ExecuteHeader)

Ports 1812 and 1813 match the standard RADIUS Authentication and Accounting port numbers used in real deployments.

5.2. Custom Packet Headers

Each protocol message is implemented as a custom ns-3 Header class with `Serialize()`, `Deserialize()`, and `GetSerializedSize()` methods.

Header	Purpose
EapHeader	Carries EAP identity, credential proof, and authentication results.
EapolHeader	Encapsulates EAP between the Supplicant and the Authenticator.
RadiusAuthHeader	Carries the authentication request from the Authenticator to RADIUS.
RadiusReplyHeader	Returns the RADIUS Access-Accept or Access-Reject decision.
LdapQueryHeader	Sends identity and credential-verification requests from RADIUS to LDAP.
LdapReplyHeader	Returns account status, role, permissions, and credential-verification results.
RadiusAcctHeader	Carries RADIUS Accounting Start and Stop messages.
TicketRequestHeader	Requests a Kerberos-inspired ticket after successful authorization.
TicketResponseHeader	Carries the issued ticket, session key, role, permissions, and expiry time.
CommandRequestHeader	Carries the command, Command MAC, Sequence Number, role, and permission claims.
ExecuteHeader	Carries an approved command from the Gateway to the target actuator.

All headers implement `Serialize()`, `Deserialize()`, and `GetSerializedSize()` for packet transmission inside ns-3.

6. Authentication and Authorization Workflow

6.1. Nonce generation

Each authentication attempt uses a fresh nonce generated from the current simulation time and the principal ID:

```
m_nonce = uint32_t(Simulator::Now().GetNanoSeconds() & 0xfffffffffu) ^ m_principalId;
```

6.2. Salted password hash and credential proof

Each LDAP entry stores a salted password hash generated by `SimplePasswordHash(secret, salt)`. During authentication, the Supplicant generates a credential proof using:

```
SimpleCredentialProof(passwordHash, nonce)
```

The password itself is not transmitted.

6.3. EAP / EAPOL exchange

`SmartHomeUserApp::SendEapolRequest()` places the principal ID, nonce, and credential proof inside an `EapHeader`, wraps it with `EapolHeader`, and sends it to the IEEE 802.1X Authenticator.

6.4. EAP Encapsulation inside RADIUS

The Authenticator removes the EAPOL wrapper and encapsulates the EAP message inside a RADIUS request. It also generates a lightweight RADIUS authenticator using the shared secret before forwarding the request to the RADIUS server.

6.5. LDAP and LDAP Cache

RADIUS verifies the request and obtains the account status, role, permissions, and credential-verification result from LDAP.

When LDAP Cache is enabled and a cached entry exists, RADIUS rechecks the credential proof using the cached password hash and avoids an additional LDAP query. The cache evaluation is presented in **Section 8**.

6.6. Access Decision and Authenticator Enforcement

RADIUS returns either `Access-Accept` or `Access-Reject`. The IEEE 802.1X Authenticator then applies this decision by setting the principal's state to:

AUTHORIZED

or:

UNAUTHORIZED

It returns EAP-Success or EAP-Failure to the Supplicant inside EAPOL.

6.7. Kerberos-Inspired Ticket Issuing

Only an authorized principal can request a ticket. The KDC verifies the IEEE 802.1X authorization state and confirms that the submitted session ID matches the current authenticated RADIUS session before issuing the ticket.

6.8. RADIUS Accounting

After receiving a ticket, the principal sends an Accounting Start message. An Accounting Stop message is sent shortly before the application ends. Each message is sent at most once, and rejected principals such as Lina send neither Start nor Stop.

6.9. Normal Scenario Results

Metric	Value	Why it is correct
Accepted	5	Ahmad, Sara, Omar, Temperature Sensor, and Motion Sensor are all enabled=true in the LDAP directory with a matching credential proof.
Rejected	1	Lina exists in the LDAP directory, but her account is disabled; therefore, RADIUS rejects her authentication request.
Authorized	5	The Authenticator enforces the five RADIUS Access-Accept decisions as AUTHORIZED IEEE 802.1X states.
Unauthorized	1	The Authenticator enforces Lina's RADIUS Access-Reject decision as the UNAUTHORIZED IEEE 802.1X state.
Accounting Starts	5	One Start per principal that actually received a ticket (guarded by m_accountingStarted); Lina never reaches this point.
Accounting Stops	5	One Stop per principal that previously sent a Start; never sent otherwise.
Tickets Issued	5	KdcApp issues a ticket only to an authorized principal with a matching RADIUS session — exactly the 5 accepted principals.
Commands Accepted	6	Ahmad: OPEN_DOOR + CAMERA_VIEW (2). Sara: LIGHT_ON (1). Omar: PLUG_ON (1). Temperature Sensor: SEND_TEMPERATURE (1). Motion Sensor: SEND_MOTION (1). Total = 6.
Commands Rejected	1	Sara's second command, OPEN_DOOR, is denied by RBAC (see next row).
RBAC Denied	1	Sara's role (GUEST) carries only the LIGHT permission bit; OPEN_DOOR requires the DOOR bit, which her permission mask does not include.

The five accepted principals are Ahmad, Sara, Omar, the Temperature Sensor, and the Motion Sensor. Lina is rejected because her LDAP account is disabled. Six commands are accepted, while Sara's OPEN_DOOR attempt is denied by RBAC because her GUEST role includes only the LIGHT permission.

7. Gateway Security Pipeline

Every command received by the Smart-Home Gateway passes through a fixed security pipeline before reaching the target IoT device.

1. **Ticket Validation** verifies that the Kerberos-inspired ticket is valid and belongs to the requesting principal.

2. **Command MAC Verification** checks the integrity of the command and detects any modification of the role or permission information.
3. **Sequence Number Verification** detects replayed packets when Replay Protection is enabled.
4. **Role-Based Access Control (RBAC)** verifies that the authenticated principal has permission to execute the requested command.

If all security checks succeed, the Gateway records the accepted sequence number and forwards the command to the target IoT device. Otherwise, the command is immediately rejected.

8. LDAP Cache Evaluation

The LDAP Cache Test uses three repeated authentication requests to measure the effect of the RADIUS-side cache. These test requests are isolated from the main IEEE 802.1X workflow and do not request Kerberos tickets, send commands, or generate RADIUS Accounting messages.

Measured comparison

Metric	Cache Disabled	Cache Enabled
Total Authentication Requests	9	9
LDAP Queries	9	7
Cache Hits	0	2
Cache Misses	0	7
LDAP Queries Saved	0	2

The nine requests consist of six normal authentication requests and three LDAP Cache Test requests. With the cache disabled, all nine requests reach LDAP. With the cache enabled, the second and third repeated test requests are served from the cache, saving two LDAP queries.

The cache does not change the authentication decision. On every cache hit, RADIUS still verifies the credential proof using the cached password hash before accepting the request.

9. Replay Attack Scenario

The Replay Attack resends a previously accepted Ahmad `OPEN_DOOR` packet without modifying it. Because the packet remains unchanged, the Kerberos-inspired ticket and Command MAC are still valid.

The Gateway detects the attack by checking the command's Sequence Number. Since the same Sequence Number was already accepted for Ahmad's current session, the packet is identified as a replay.

Results

Metric	Protection OFF	Protection ON
Attempts	1	1
Detected	1	1
Accepted	1	0
Blocked	0	1

With Replay Protection disabled, the duplicated command is detected but still accepted and executed again. With Replay Protection enabled, the Gateway blocks the packet before forwarding it to the Smart Door Lock.

10. Privilege Escalation Scenario

Sara is legitimately authenticated as `GUEST` with the `LIGHT` permission only. During the Privilege Escalation scenario, the `role` and `permission` fields in her command are changed to `OWNER` and `DOOR` after the Command MAC has already been calculated using her original privileges.

The Kerberos-inspired ticket is not modified. At the Gateway, the Command MAC is recalculated using the received forged values, causing a mismatch that identifies the attack.

Results

Metric	Protection OFF	Protection ON
Attempts	1	1
Detected	1	1
Accepted	1	0
Blocked	0	1

With protection disabled, the forged command is detected but still accepted and reaches the Smart Door Lock. With protection enabled, the Gateway blocks the command before RBAC validation and device execution.

11. Build and Execution Commands

All commands assume the file is placed at `scratch/iot-defense-demo.cc` inside the `ns-3.41` source tree.

Build

```
cd ~/iot-security-thesis/ns-allinone-3.41/ns-3.41
./ns3 build
```

Normal scenario

```
./ns3 run "scratch/iot-defense-demo.cc --showAuthenticationFlow=1"
```

LDAP Cache

```
./ns3 run "scratch/iot-defense-demo.cc --showAuthenticationFlow=0 --enableLdapCache=1 --enableCacheDemo=1"
```

```
# Cache-disabled comparison run (for the Section 8 table)
./ns3 run "scratch/iot-defense-demo.cc --showAuthenticationFlow=0 --enableLdapCache=0 --enableCacheDemo=1"
```

Replay Attack — protection OFF and ON

```
./ns3 run "scratch/iot-defense-demo.cc --showAuthenticationFlow=0 --enableReplayAttack=1 --enableReplayProtection=0"
./ns3 run "scratch/iot-defense-demo.cc --showAuthenticationFlow=0 --enableReplayAttack=1 --enableReplayProtection=1"
```

Privilege Escalation — protection OFF and ON

```
./ns3 run "scratch/iot-defense-demo.cc --showAuthenticationFlow=0 --enablePrivilegeEscalationAttack=1 --enablePrivilegeEscalationProtection=0"
./ns3 run "scratch/iot-defense-demo.cc --showAuthenticationFlow=0 --enablePrivilegeEscalationAttack=1 --enablePrivilegeEscalationProtection=1"
```

Full protected scenario

```
./ns3 run "scratch/iot-defense-demo.cc --showAuthenticationFlow=0 --enableLdapCache=1 --enableCacheDemo=1 --enableReplayAttack=1 --enableReplayProtection=1 --enablePrivilegeEscalationAttack=1 --enablePrivilegeEscalationProtection=1"
```

12. Code Defense Index

The source file `iot-defense-demo.cc` contains searchable `// SHOW:` labels placed above the main implementation sections. These labels help during the defense by directly locating the required security mechanisms.

Search Label	Class / Function	What It Proves to the Committee
SHOW: IEEE 802.1X – AUTHENTICATOR	Dot1xAuthenticatorApp	Network access control exists as its own node, separate from RADIUS and the Gateway.
SHOW: EAPOL – REQUEST	SmartHomeUserApp::SendEapolRequest	EAP is correctly wrapped inside EAPOL on the Supplicant side.
SHOW: RADIUS – AUTHENTICATION	RadiusServerApp::ReceiveAuth	RADIUS performs the actual authentication decision, not the Authenticator.
SHOW: RADIUS – ACCOUNTING	RadiusServerApp::ReceiveAcct	A dedicated Accounting port/handler exists, separate from Authentication.
SHOW: LDAP – LOOKUP	LdapServerApp::ReceiveQuery	Identity, role, and permission resolution is delegated to a separate LDAP application node.
SHOW: LDAP CACHE – HIT AND MISS	RadiusServerApp::ReceiveAuth (cache branch)	The cache is implemented at the RADIUS layer and re-

Search Label	Class / Function	What It Proves to the Committee
		verifies credentials on every hit.
SHOW: KERBEROS – TICKET ISSUING	KdcApp::IssueKerberosTicket	Tickets are issued only after authorization and RADIUS-session verification.
SHOW: KERBEROS – TICKET VALIDATION	SmartHomeGatewayApp::ValidateTicket	The Gateway independently re-validates the ticket on every command.
SHOW: COMMAND MAC – GENERATION	CalculateCommandMac	A single, reusable MAC function covers session key, principal, device, command, timestamp, sequence number, role, and permission.
SHOW: COMMAND MAC – VERIFICATION	SmartHomeGatewayApp::ValidateCommandMac	The Gateway recomputes the MAC independently rather than trusting the sender.
SHOW: SEQUENCE NUMBER – GENERATION	SmartHomeUserApp::CreateCommandPacket	Every command carries a monotonically increasing sequence number.
SHOW: REPLAY ATTACK – PACKET CAPTURE	SmartHomeGatewayApp::CaptureLegitimateReplayPacket	The captured packet is a real, previously accepted packet — not a synthetic one.
SHOW: REPLAY ATTACK – PACKET RESEND	ReplayAttackApp::TrySendReplay	The resent packet is byte-identical to the original, proving the attack is a true replay.
SHOW: ANTI-REPLAY – DUPLICATE DETECTION	SmartHomeGatewayApp::DetectReplay	Detection is based solely on sequence-number duplication.
SHOW: RBAC – PERMISSION CHECK	SmartHomeGatewayApp::ValidatePermission	RBAC is enforced as a distinct, final stage after the MAC and replay checks.
SHOW: PRIVILEGE ESCALATION – FORGED CLAIMS	SmartHomeUserApp::CreatePrivilegeEscalationAttempt	The attack forges command claims only; the ticket is provably untouched.
SHOW: PRIVILEGE ESCALATION – DETECTION AND BLOCKING	SmartHomeGatewayApp::ValidateCommandMac	Detection reuses the same Command MAC mechanism — no separate detection path exists.
SHOW: GATEWAY – SECURITY PIPELINE	SmartHomeGatewayApp::ProcessGatewayCommand	The full, fixed-order enforcement pipeline is visible in one place.

Search Label	Class / Function	What It Proves to the Committee
SHOW: FINAL RESULTS	PrintResults()	Locates the final simulation counters and scenario results.

13. Main Code Excerpts

The following simplified excerpts highlight the main security mechanisms. Some parameters and nonessential lines are omitted for readability. The complete implementation remains available in `iot-defense-demo.cc`.

13.1. EAP inside EAPOL

SmartHomeUserApp::SendEapolRequest — the Supplicant wraps EAP inside EAPOL:

```
EapHeader eap(...);
EapolHeader eapol;
Ptr<Packet> p = Create<Packet>(0);
p->AddHeader(eap);
p->AddHeader(eapol);
```

13.2. EAP inside RADIUS

Dot1xAuthenticatorApp::ReceiveEapol — the Authenticator re-encapsulates the same EAP payload inside RADIUS:

```
RadiusAuthHeader a(...);
Ptr<Packet> out = Create<Packet>(0);
out->AddHeader(eap);
out->AddHeader(a);
```

13.3. LDAP Cache

```
if (m_enableCache) {
    auto cit = m_cache.find(a.GetPrincipalId());
    if (cit != m_cache.end()) {
        g_res.cacheHits++;
        ...
        return;
    }
}
```

```
g_res.cacheMisses++;
}
```

13.4. Kerberos Ticket Issuing

```
if (!m_authenticator->IsAuthorized(...)) return;
if (tr.GetSessionId() != m_radius->GetSessionId(...)) return;

TicketResponseHeader tk(...);
```

13.5. Gateway Security Pipeline

```
if (!ValidateTicket(...)) return;
if (!ValidateCommandMac(...)) return;
if (!DetectReplay(...)) return;
if (!ValidatePermission(...)) return;

StoreAcceptedSequence(...);
ForwardToDevice(...);
```

13.6. Replay and Privilege Escalation Protection

Replay Detection

```
alreadyAccepted =
    m_acceptedSequences[sk].count(cmdReq.GetSeq()) > 0;

if (m_enableReplayProtection)
{
    g_res.dropReplay++;
    return false;
}
```

Privilege Escalation Detection

```
bool claimsMismatch =
    (cmdReq.GetRole() != ticket.GetRoleId()) ||
    (cmdReq.GetPerm() != ticket.GetPermissionMask());
```

```
if (claimsMismatch && !macValid)
{
    return HandlePrivilegeEscalation(cmdReq);
}
```

Implementation Scope

The project functionally models IEEE 802.1X, EAP/EAPOL, RADIUS, LDAP, Kerberos-inspired ticketing, RBAC, Command MAC verification, and replay protection at the ns-3 application layer. The implemented hash and MAC functions are lightweight simulation models and are not production-grade cryptographic algorithms. The objective is to evaluate security behavior and compare protection-enabled and protection-disabled scenarios within a controlled ns-3 environment.

Final Verification

The implementation contains exactly two attack scenarios: Replay Attack and Privilege Escalation Attack. The Gateway security pipeline, LDAP Cache behavior, RADIUS Accounting, Command MAC verification, Sequence Number replay detection, and RBAC enforcement were verified against the final `iot-defense-demo.cc` implementation.